



РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет(Сопроводительная записка)

По учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся Миниханов Вадим Павлович
(фамилия, имя, отчество)

Группы ИПс-21

Руководитель практической подготовки от профильной организации
Преподаватель
(должность)

Берестов Дмитрий Борисович

МП

«26» июня 2026 года

Пермь, 2026 год

Оглавление

ВВЕДЕНИЕ	3
1. Структура проекта.....	4
2. Основные зависимости	6
3. Интерфейс программы.....	7
3.1. Главное окно (MainWindow)	7
3.2. Элементы управления.....	8
3.3. Окно добавления и редактирования задачи (AddEditTaskWindow)	10
4. Логическая структура программы (C#)	12
4.1. Модель данных класс TaskItem.....	12
4.2. Интерфейс ITaskService	13
4.3. Реализация TaskService.....	13
4.4. Вспомогательный класс JsonHelper	16
5. Методы и обработчики событий	17
5.1. Конструктор и инициализация	17
5.2. Метод RefreshTaskList.....	17
5.3. Обработчики кнопок	18
5.4. Логика окна AddEditTaskWindow	19
6. Тестирование и отладка.....	20
6.1. Тестовые случаи для проверки корректности работы	20
6.2. Возможные ошибки и исключения.....	22
Заключение.....	25

ВВЕДЕНИЕ

Этот отчёт — о том, как разрабатывался «Менеджер задач», мой планировщик дел, который я делал во время производственной практики. Здесь я рассказываю, как устроено приложение: какую архитектуру выбрал, какие технологии использовал, как выглядит интерфейс, как работают основные модули, и что из этого в итоге получилось.

Готовая программа — это обычное десктопное приложение для Windows, которое помогает управлять задачами — как рабочими, так и личными. С его помощью можно:

- создавать задачи с описанием, приоритетом и дедлайном;
- следить, на каком они этапе;
- быстро искать нужное по ключевым словам и фильтровать список;
- сортировать дела по важности или по сроку;
- сохранять всё в JSON-файл и при необходимости загружать обратно.

Писалась программа на C#, а интерфейс сделан на WPF. Чтобы логика не перемешивалась с отображением, я вынес всю бизнес-суть в отдельную библиотеку классов — так удобнее тестировать и проще поддерживать код.

Дальше по отчёту я по шагам прошёл весь путь: от разбора задачи и составления техзадания до отладки, тестов и проверки кода. Надеюсь, по этим страницам будет видно, как рождалось решение и почему я принимал те или иные проектные решения.

1. Структура проекта

Программный продукт реализован в виде решения (Solution), состоящего из трёх самостоятельных проектов, каждый из которых выполняет строго определённую роль в архитектуре приложения. Подобное разделение соответствует принципу единственной ответственности и обеспечивает высокую степень сопровождаемости программного кода.

Проект	Тип	Назначение
Minihanov.Praktik.Core	Библиотека классов (.dll)	Содержит бизнес-логику приложения: модели данных, интерфейсы, сервисы и вспомогательные классы. Не содержит зависимостей от WPF.
Minihanov.Praktik1	WPF-приложение (.exe)	Исполняемый проект с графическим интерфейсом пользователя. Содержит XAML-разметку и код-behind для окон приложения.
Minihanov.Praktik1.Tests	Проект тестов MSTest (.dll)	Содержит модульные тесты, покрывающие бизнес-логику библиотеки Core независимо от слоя представления.

Иерархия файлов проекта в среде разработки Visual Studio 2026 представлена на рисунке 1.

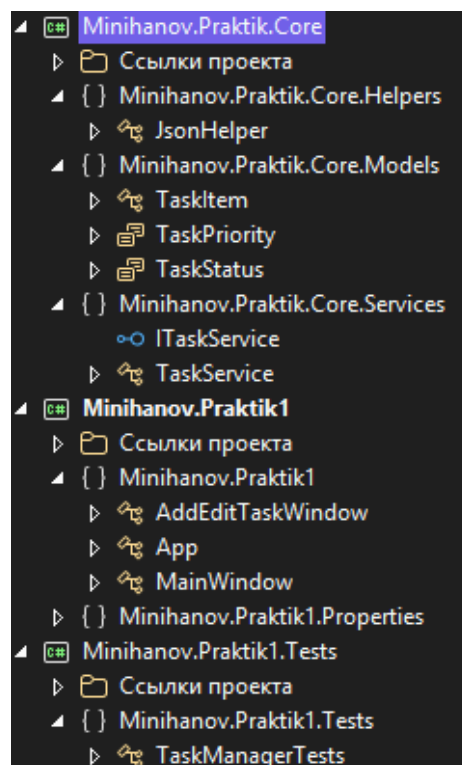


Рисунок 1 Представление классов

Библиотека `Minihanov.Praktik.Core` устроена просто: внутри неё три логические папки, каждая отвечает за своё дело.

В `Helpers` лежат вспомогательные утилиты — например, `JsonHelper`, который занимается сохранением и загрузкой данных.

В `Models` хранятся модели данных и перечисления, то есть описание того, из чего состоит задача (`TaskItem`) и какие у неё могут быть статусы и приоритеты.

А в `Services` — сама логика работы с задачами: интерфейс `ITaskService` и его реализация `TaskService`, через которые выполняются все операции.

Что касается самого интерфейса — проект `Minihanov.Praktik1` — там выделена отдельная папка `Views`. В неё я сложил два окна, с которыми работает пользователь: главное `MainWindow` и диалоговое окошко для добавления и редактирования задач `AddEditTaskWindow`. Всё разложено по полочкам, чтобы было понятно, где что искать.

2. Основные зависимости

В проекте я использовал минимум внешних библиотек — всё, что нужно, либо уже есть в .NET, либо подтягивается стандартными средствами. Из дополнительного — только фреймворк для тестов MSTest. Всё остальное родные инструменты платформы.

Вот кратко, какая библиотека за что отвечает:

Зависимость	Где используется	Зачем нужна
System.Text.Json	JsonHelper.cs	Превращает объекты в JSON и обратно. Встроена в .NET 6+, ничего ставить не пришлось.
System.ComponentModel	TaskItem.cs	Даёт интерфейс INotifyPropertyChanged, чтобы интерфейс автоматически обновлялся при изменении данных.
System.Collections.Generic	TaskService.cs	Обычный список List<T>, в котором задачи хранятся в памяти.
System.Linq	TaskService.cs, MainWindow.xaml.cs	LINQ-запросы — с их помощью фильтрую, сортирую и ищу задачи.
Microsoft.Win32	MainWindow.xaml.cs	Стандартные диалоги для открытия и сохранения файлов.
MSTest Framework	TaskManagerTests.cs	Сам фреймворк для модульных тестов. Атрибуты [TestClass], [TestMethod], методы проверок Assert.
System.IO	JsonHelper.cs, тесты	Чтение и запись файлов: File.ReadAllText, File.WriteAllText, File.Exists и другие.

Самое важное архитектурное решение в библиотеке Core нет ни одной привязки к WPF. Это сделано сознательно: бизнес-логику можно тестировать отдельно, без запуска интерфейса, а в будущем при необходимости даже переиспользовать библиотеку в других типах проектов, например в консольном приложении или веб-сервисе

3. Интерфейс программы

Графический интерфейс пользователя реализован с применением технологии Windows Presentation Foundation (WPF) и описан посредством языка разметки XAML. Приложение включает два окна: главное окно и диалоговое окно добавления и редактирования задачи.

3.1. Главное окно (MainWindow)

Главное окно является основным рабочим экраном приложения. В качестве корневого контейнера выступает элемент Grid с пятью строками, каждая из которых выполняет определённую функцию в компоновке интерфейса. Общий вид главного окна представлен на рисунке 2.

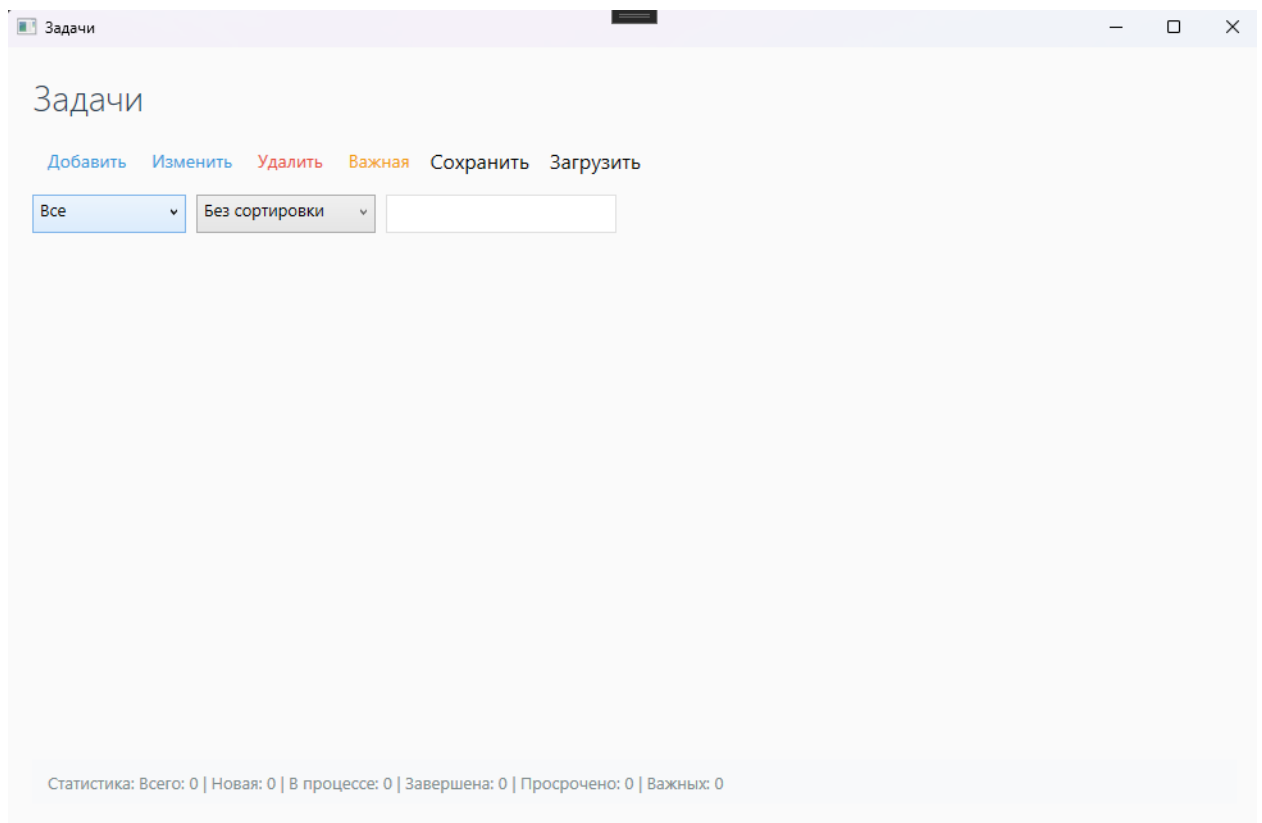


Рисунок 2 Главное окно приложения «Планировщик задач»

Для размещения элементов на главном окне я использовал таблицу-сетку Grid. Вот как она выглядит:

```
<Grid Margin="20">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/> <!-- строка заголовка -->
    <RowDefinition Height="Auto"/> <!-- панель кнопок -->
    <RowDefinition Height="Auto"/> <!-- панель фильтров -->
    <RowDefinition Height="*/> <!-- список задач -->
    <RowDefinition Height="Auto"/> <!-- строка статистики-->
```

```
</Grid.RowDefinitions>  
</Grid>
```

Здесь каждая строка отвечает за свой блок. Для строки со списком задач я указал `Height="*"` это значит, что она будет занимать всё оставшееся место по вертикали, как бы растягиваясь под размер окна. А все остальные строки имеют высоту `Auto` они подстраиваются под содержимое: сколько места нужно кнопкам, фильтрам или заголовку, столько они и займут.

3.2. Элементы управления

Заголовок и часы

В самой верхней строке находится горизонтальный контейнер `StackPanel`. Внутри него обычный заголовок окна и текстовый блок `txtDateTime`, который показывает текущие дату и время. Чтобы время обновлялось каждую секунду, я задействовал `DispatcherTimer` он тикает в фоновом режиме и обновляет значение на экране.

Панель кнопок

Следом идёт панель с основными кнопками управления это `WrapPanel`, где кнопки аккуратно выстраиваются в ряд. Все они оформлены в минималистичном духе: прозрачный фон, цветной текст и никаких рамок, чтобы не отвлекать внимание. А вот что делает каждая из них их шесть, и у каждой своя задача. Я постарался сделать их интуитивно понятными, а цвета подобрал так, чтобы они несли смысловую нагрузку.

Добавить и Изменить (`btnAdd`, `btnEdit`) обе синие. При нажатии на «Добавить» открывается пустое диалоговое окно для создания новой задачи. Кнопка «Изменить» работает только когда в списке что-то выбрано она открывает то же окно, но уже с заполненными полями выбранной задачи.

Удалить (`btnDelete`) красная, чтобы сразу привлекала внимание. Это опасное действие, поэтому перед удалением всегда появляется диалог с подтверждением случайно не сработает.

Важная (btnImportant) оранжевая .Выделяет задачу как важную: нажимаете появляется звёздочка, нажимаете ещё раз звёздочка исчезает. Просто как переключатель.

Сохранить и Загрузить (btnSave, btnLoad) обе чёрные . Первая экспортирует текущий список задач в JSON-файл, вторая импортирует ранее сохранённые задачи. В обоих случаях появляется диалог выбора папки и имени файла, так что пользователь сам решает, где и под каким именем хранить свои данные.

Панель фильтров и строка поиска

Третья строка интерфейса это, по сути, командный центр для работы со списком задач. Здесь собраны все инструменты, которые помогают не утонуть в делах: фильтрация, сортировка и поиск.

Слева расположен выпадающий список cmbStatusFilter он позволяет быстро отсеять всё лишнее и показать задачи только с определённым статусом, например только новые или только завершённые.

Рядом с ним ещё один выпадающий список cmbSort, который отвечает за сортировку. С его помощью можно упорядочить задачи по приоритету или по сроку выполнения, чтобы самое важное или самое срочное всегда было наверху.

И наконец, текстовое поле txtSearch это полноценный поиск по названию и описанию задачи. Вводите ключевое слово список мгновенно сужается до подходящих вариантов.

Все три инструмента работают вместе: можно одновременно отфильтровать задачи по статусу, отсортировать их по сроку и при этом ещё что-то найти по ключевому слову. Внешний вид этой панели показан на рисунке 3.

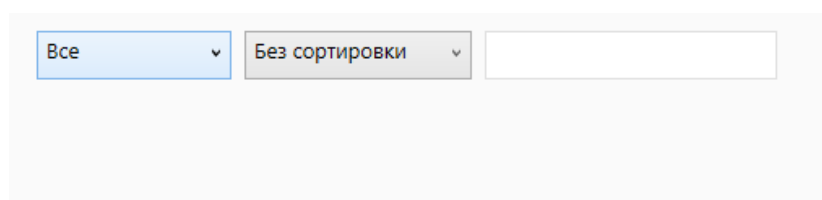


Рисунок 3 Панель фильтрации и поиска

Четвёртая строка это сердце всего приложения. Здесь находится элемент `ListBox`, который выводит все задачи в виде аккуратных карточек. Каждая карточка формируется по шаблону `DataTemplate`, так что все задачи выглядят единообразно и при этом содержат всю нужную информацию.

На карточке сразу видно самое главное: название задачи, краткое описание, текущий статус, приоритет, срок выполнения. А ещё есть пара визуальных подсказок звёздочка, если задача отмечена как важная, и особое выделение, если срок уже прошёл. Всё это помогает быстро оценить ситуацию, не открывая каждую задачу по отдельности.

Как это выглядит в реальности можно посмотреть на рисунке 4.

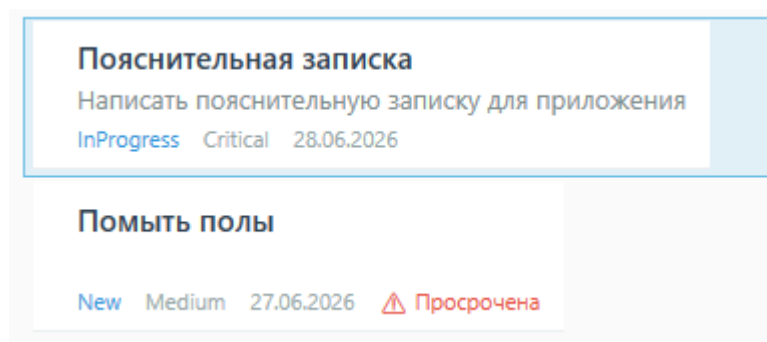


Рисунок 4 Список задач с карточками

Чтобы визуально показывать, просрочена задача или нет, я использовал стандартный конвертер `BooleanToVisibilityConverter`. Работает он просто: берёт булево свойство `IsOverdue` и автоматически решает, показывать соответствующий индикатор на карточке или скрывать. Никакой ручной логики встроенный механизм сам преобразует `true` в видимый элемент, а `false` в скрытый.

3.3. Окно добавления и редактирования задачи (`AddEditTaskWindow`)

Это диалоговое окно появляется поверх главного в модальном режиме — то есть пока оно открыто, пользователь не может переключиться на основное окно. Вызывается оно кнопками «Добавить» или «Изменить».

Внутри — аккуратная сетка `Grid` с двумя колонками. Слева расположены названия полей (метки), а справа — сами поля для ввода: текстовые поля, выпадающие списки и календарик для выбора даты. Всё логично и понятно, никакой путаницы. Как это выглядит можно посмотреть на рисунке 5.

Детали задачи

Название:

Описание:

Приоритет: Средний

Срок выполнения: 28.06.2026

Статус: Новая

☐ ★ Важная задача

OK Отмена

Рисунок 5 — Диалоговое окно добавления / редактирования задачи

Поля формы добавления и редактирования

В диалоговом окне собраны все необходимые поля для работы с задачей. Каждое из них решает свою задачу, а вместе они дают полную картину того, что нужно сделать.

Название (`txtTitle`) — обязательное текстовое поле. Без него задачу не сохранить, так что это первое, что заполняет пользователь.

Описание (`txtDescription`) — многострочное текстовое поле. Здесь можно написать подробности, пояснения или любую дополнительную информацию. Поддерживается перенос строк, так что описание не превращается в кашу.

Приоритет (`cmbPriority`) — выпадающий список с четырьмя вариантами: Низкий, Средний, Высокий, Критический. Помогает расставить дела по степени важности.

Срок выполнения (`dpDueDate`) — календарик для выбора даты. Удобно и наглядно, никакого ручного ввода.

Статус (cmbStatus) — ещё один выпадающий список: Новая, В процессе, Завершена. Позволяет отслеживать, на каком этапе находится задача.

Важная (chkImportant) — простой флажок-галочка. Отмечаете — задача становится важной и в общем списке выделяется звёздочкой.

Внизу формы — две кнопки:

Сохранить (btnOk) — проверяет, заполнено ли название, и если всё в порядке, сохраняет задачу, а затем закрывает окно.

Отмена (btnCancel) — просто закрывает окно без сохранения, ничего не меняя.

4. Логическая структура программы (C#)

Теперь давайте заглянем внутрь кода разберём, из чего состоит приложение, как связаны между собой классы и почему я принял те или иные решения.

4.1. Модель данных класс TaskItem

Всё начинается с класса TaskItem. Это главная модель данных, которая описывает, из чего состоит задача. Он реализует два полезных интерфейса:

INotifyPropertyChanged чтобы при изменении любого свойства интерфейс на WPF автоматически обновлялся без лишних телодвижений.

ICloneable чтобы можно было создавать независимые копии объектов.

Каждое свойство в классе при изменении вызывает метод OnPropertyChanged, который шлёт уведомление интерфейсу: «Внимание, данные поменялись, обновляй отображение!».

Вычисляемое свойство IsOverdue — оно не хранится в памяти, а вычисляется на лету по формуле:

```
public bool IsOverdue =>
```

```
Status != TaskStatus.Completed && DueDate.Date < DateTime.Today;
```

Здесь важно, что я сравниваю только дату, без времени. Это сделано специально: если срок задачи сегодняшний день, она не считается просроченной до самого вечера. Задача с дедлайном на сегодня ещё актуальна.

Метод Clone() реализует защитное копирование. Создаёт новый объект с теми же полями:

```
public object Clone()
{
    return new TaskItem {
        Id = this.Id, Title = this.Title,
        Description = this.Description,
        Priority = this.Priority, DueDate = this.DueDate,
        Status = this.Status, IsImportant = this.IsImportant
    };
}
```

Зачем это нужно? Когда я возвращаю задачи из сервиса, я отдаю не оригиналы, а их копии. Это предотвращает случайные изменения извне — кто-то получит задачу, изменит её, а оригинал в коллекции останется нетронутым. Страховка от неожиданных побочных эффектов.

4.2. Интерфейс `ITaskService`

Интерфейс `ITaskService` — это не просто формальность. Он задаёт чёткий контракт: какие операции доступны для работы с задачами. В главном окне я никогда не обращаюсь напрямую к конкретной реализации, всегда только через интерфейс.

Такой подход даёт слабую связанность: если завтра я захочу заменить способ хранения данных (например, перейти с памяти на базу данных), мне достаточно будет написать новый класс, реализующий тот же интерфейс, и подменить его в одном месте. Всё остальное продолжит работать. Всего в интерфейсе 13 методов — полный набор CRUD-операций плюс поиск, фильтрация, сортировка, работа с файлами и статистика.

4.3. Реализация `TaskService`

Класс `TaskService` — это та самая реализация, которая непосредственно работает с задачами. Данные хранятся в оперативной памяти в обычном списке `List<TaskItem>`.

Поскольку к списку могут одновременно обращаться разные части программы, я добавил потокобезопасность — все операции, которые изменяют или читают коллекцию, обёрнуты в блок `lock (_lockObject)`. Это гарантирует, что в любой момент времени только один поток может работать с данными.

Метод `AddTask` — добавление задачи:

```
public void AddTask(TaskItem task)
{
    if (task == null)
        throw new ArgumentNullException(nameof(task));
    if (string.IsNullOrEmpty(task.Title))
        throw new ArgumentException("Название не может быть пустым");
    lock (_lockObject)
    {
        task.Id = Guid.NewGuid().ToString();
        _tasks.Add((TaskItem)task.Clone());
    }
}
```

Сначала я проверяю, что задача не пустая и у неё есть название — без этого сохранять бессмысленно. Затем генерирую уникальный идентификатор через `Guid` и добавляю в список не сам переданный объект, а его клон. Опять защита от внешних изменений.

Метод `SearchTasks` — поиск по ключевому слову:

```
public IEnumerable<TaskItem> SearchTasks(string searchTerm)
{
    if (string.IsNullOrEmpty(searchTerm))
        return GetAllTasks();

    lock (_lockObject)
    {
```

```

        return _tasks.Where(t =>
            t.Title.IndexOf(searchTerm, StringComparison.OrdinalIgnoreCase)
            >= 0 ||
            (t.Description != null && t.Description.IndexOf(searchTerm,
                StringComparison.OrdinalIgnoreCase) >= 0))
            .Select(t => (TaskItem)t.Clone())
            .ToList();
    }
}

```

Здесь всё просто: ищу совпадения в названии или описании, причём без учёта регистра (OrdinalIgnoreCase). То есть пользователь может ввести «важн» и найдёт задачу «Важная встреча» — удобно.

Метод GetStatistics — статистика по задачам:

```

public (int Total, int New, int InProgress, int Completed, int Overdue, int
Important) GetStatistics()
{
    lock (_lockObject)
    {
        var total = _tasks.Count;
        var    newTasks    =    _tasks.Count(t    =>    t.Status    ==
Minihanov.Praktik.Core.Models.TaskStatus.New);
        var    inProgress    =    _tasks.Count(t    =>    t.Status    ==
Minihanov.Praktik.Core.Models.TaskStatus.InProgress);
        var    completed    =    _tasks.Count(t    =>    t.Status    ==
Minihanov.Praktik.Core.Models.TaskStatus.Completed);
        var overdue = _tasks.Count(t => t.IsOverdue);
        var important = _tasks.Count(t => t.IsImportant);

        return (total, newTasks, inProgress, completed, overdue, important);
    }
}

```

```
}
```

Вместо того чтобы создавать отдельный класс для результатов, я использовал именованный кортеж. Это коротко и удобно: возвращаю несколько значений одной пачкой, и каждое имеет имя `Total`, `New`, `Completed` и так далее.

4.4. Вспомогательный класс `JsonHelper`

`JsonHelper` это статический вспомогательный класс, который отвечает за сохранение и загрузку данных. Он работает через стандартную библиотеку `System.Text.Json` лёгкую и быструю.

Я настроил сериализатор так, чтобы JSON выглядел аккуратно и читабельно:

```
private static readonly JsonSerializerOptions Options = new
JsonSerializerOptions
{
    WriteIndented = true,
    PropertyNameCaseInsensitive = true,
    Converters = { new JsonStringEnumConverter() }
};
```

- `WriteIndented = true` — добавляет отступы, так что файл можно открыть в блокноте и спокойно править руками.

- `PropertyNameCaseInsensitive = true` — при загрузке неважно, заглавные буквы или строчные, конвертер разберётся.

- `JsonStringEnumConverter` — ключевая настройка. Перечисления (`TaskStatus`, `TaskPriority`) сохраняются не как числа (0, 1, 2), а как понятные строки — `"New"`, `"High"`, `"Completed"`. Это делает JSON-файл удобным для человека: можно прочитать и при необходимости отредактировать вручную.

5. Методы и обработчики событий

5.1. Конструктор и инициализация

Всё начинается с конструктора главного окна. Вот как он выглядит:

```
csharp
public MainWindow()
{
    _taskService = new TaskService();
    InitializeComponent();
    _isInitialized = true;
    RefreshTaskList();
}
```

Последовательность здесь не случайна. Сначала я создаю экземпляр сервиса он нужен, чтобы работать с задачами с самого начала. Потом вызываю `InitializeComponent()`, который загружает и рисует весь интерфейс. И только после того как все кнопки, списки и поля готовы, я устанавливаю флаг `_isInitialized = true` и вызываю обновление списка задач.

Флаг `_isInitialized` это защита от дурака. Дело в том, что некоторые обработчики событий могут сработать в процессе инициализации, когда интерфейс ещё не полностью построен. Благодаря этому флагу они проверяют: «А можно ли уже обновлять список?» и если нет, просто игнорируют вызов. Именно это решение помогло избавиться от ошибки `NullReferenceException`, которая возникала в ранней версии приложения.

5.2. Метод `RefreshTaskList`

Это, пожалуй, самый главный метод во всей логике отображения. Его задача — взять актуальные данные, применить все фильтры и сортировку, показать пользователю то, что он хочет видеть в данный момент.

Метод вызывается везде, где что-то меняется: добавили задачу, обновили, отредактировали, обновили, изменили фильтр, снова обновили.

Внутри всё происходит по этапам:

- Сначала я запрашиваю у сервиса полный список всех задач.

- Затем, если пользователь выбрал какой-то статус в выпадающем списке (не «Все»), применяю фильтр оставляю только задачи с этим статусом.
- Если в строке поиска есть текст ищу совпадения в названии и описании.
- И наконец, сортирую результат: либо по приоритету, либо по сроку, в зависимости от выбранного режима.

После всех преобразований я кладу получившийся список в элемент `lstTasks` и он мгновенно перерисовывается. В самом конце вызываю `UpdateStatistics()` чтобы строка статистики снизу тоже обновилась и показывала актуальные цифры.

5.3. Обработчики кнопок

Здесь всё довольно прямолинейно, но есть несколько нюансов.

Добавление задачи:

```
private void btnAdd_Click(object sender, RoutedEventArgs e)
{
    var addWindow = new AddEditTaskWindow();
    if (addWindow.ShowDialog() == true)
    {
        _taskService.AddTask(addWindow.TaskItem);
        RefreshTaskList();
    }
}
```

Открывается диалоговое окно. Если пользователь нажал «Сохранить» (а не «Отмена»), то я забираю созданную задачу, добавляю её в сервис и обновляю список.

Удаление здесь я добавил дополнительную страховку. Перед тем как удалить задачу, появляется диалог подтверждения: «Вы уверены?». Это защита от случайных кликов лучше один раз спросить, чем потом восстанавливать потерянные данные.

И для удаления, и для редактирования я сначала проверяю: а выбрана ли вообще какая-то задача в списке? Если нет вывожу сообщение и ничего не делаю. Пользователь не должен гадать, почему кнопка не работает.

5.4. Логика окна AddEditTaskWindow

Это диалоговое окно умеет работать в двух режимах: создание новой задачи и редактирование существующей.

В режиме редактирования я передаю в конструктор объект задачи, но не сам оригинал, а его копию через Clone(). Зачем? Чтобы пользователь мог смело менять поля, отменять изменения и исходная задача оставалась нетронутой до самого последнего момента. Только когда он нажимает «Сохранить», я забираю данные из формы и обновляю оригинал.

При открытии окна вызывается метод LoadTaskData(), который заполняет все поля: название, описание, приоритет, дату, статус и галочку «Важная». Всё подставляется автоматически, пользователю не нужно вводить заново.

При нажатии «ОК» срабатывает валидация: проверяю, что название не пустое и что дата выбрана. Если какое-то поле не заполнено показываю сообщение и не даю закрыть окно. Если всё хорошо собираю данные из полей, формирую объект задачи (или обновляю существующий), и закрываю диалог с результатом true. Главное окно, получив этот сигнал, забирает задачу и обновляет список.

6. Тестирование и отладка

6.1. Тестовые случаи для проверки корректности работы

Для тестирования я создал отдельный проект `Minihanov.Praktik1.Tests` и использовал фреймворк `MSTest`. Внутри есть класс `TaskManagerTests`, который реализует интерфейс `IDisposable`. Это сделано для того, чтобы после каждого теста автоматически подчищать временные файлы, которые создаются в процессе проверок — чтобы мусор не накапливался и не мешал следующим тестам.

Каждый тест запускается с чистым экземпляром `TaskService` — своим собственным. Это гарантирует, что тесты полностью изолированы друг от друга: данные одного не влияют на данные другого. Так я могу быть уверен, что если какой-то тест упал, то виноват именно он, а не предыдущий тест, который что-то недопочистил.

Всего я написал 16 тестов, которые покрывают практически всё, что только можно: от добавления задачи до работы с файлами и клонирования. Вот что проверяет каждый из них.

Базовые CRUD-операции

Первые три теста это основа основ. `AddTask_ShouldAddTaskSuccessfully` проверяет, что задача действительно добавляется и все поля заполняются корректно. `UpdateTask_ShouldUpdateTaskSuccessfully` что можно изменить любое поле, и изменения действительно сохраняются. А `DeleteTask_ShouldDeleteTaskSuccessfully` убеждается, что после удаления список становится пустым — ничего лишнего не остаётся.

Получение данных

Дальше идут тесты на чтение. `GetTaskById_ShouldReturnTaskSuccessfully` если задача существует, мы её находим по идентификатору. А `GetTaskById_ShouldReturnNull_WhenTaskNotFound` проверяет противоположный случай: если задачи с таким `Id` нет, метод возвращает `null`,

а не вылетает с ошибкой. `GetAllTasks_ShouldReturnAllTasks` просто убеждается, что возвращается полный список всех задач.

Фильтрация, поиск и сортировка

Здесь три важных сценария. `GetTasksByStatus_ShouldReturnCorrectTasks` фильтрация по статусу, например, показываем только задачи со статусом `New`. `SearchTasks_ShouldReturnResultsByTitle` поиск по ключевому слову в названии, работает без учёта регистра. `GetTasksSortedByPriority_ShouldReturnSortedTasks` сортировка от высокого приоритета к низкому: `High` → `Medium` → `Low`.

Работа с файлами

`LoadFromFile_ShouldLoadTasksSuccessfully` проверяет, что задача сохраняется в JSON-файл, а потом без потерь загружается обратно. Всё на месте, все поля совпадают.

Статистика и переключение флагов

`GetStatistics_ShouldReturnCorrectCounts` считает статистику: сколько новых задач, сколько в процессе, сколько завершённых, сколько просроченных и сколько важных. Все цифры должны совпадать с тем, что есть в списке. `ToggleImportant_ShouldToggleFlag` проверяет, что флаг важности переключается как надо: сначала `false`, потом `true`, потом снова `false`.

Свойства и клонирование

`IsOverdue_ShouldReturnTrue_WhenTaskIsOverdue` вычисляемое свойство просроченности: для задачи с истёкшим сроком оно должно быть `true`. `TaskItem_Constructor_ShouldInitializeDefaultValues` просто проверяет, что конструктор по умолчанию правильно выставляет значения: статус `New`, приоритет `Medium`, флаг важности `false`.

И два теста на копирование: `TaskItem_Clone_ShouldCreateDeepCopy` убеждается, что клон полностью независим от оригинала, изменения в одном не влияют на другой. И `JsonHelper_LoadFromFile_ShouldLoadDataSuccessfully` отдельно проверяет вспомогательный класс, который отвечает за сохранение и загрузку, чтобы быть уверенным, что сериализация работает корректно.

В итоге все эти тесты вместе дают уверенность, что приложение работает стабильно, а если что-то сломается тесты это сразу покажут.

Результат выполнения всех тестов в среде Visual Studio представлен на рисунке 6.

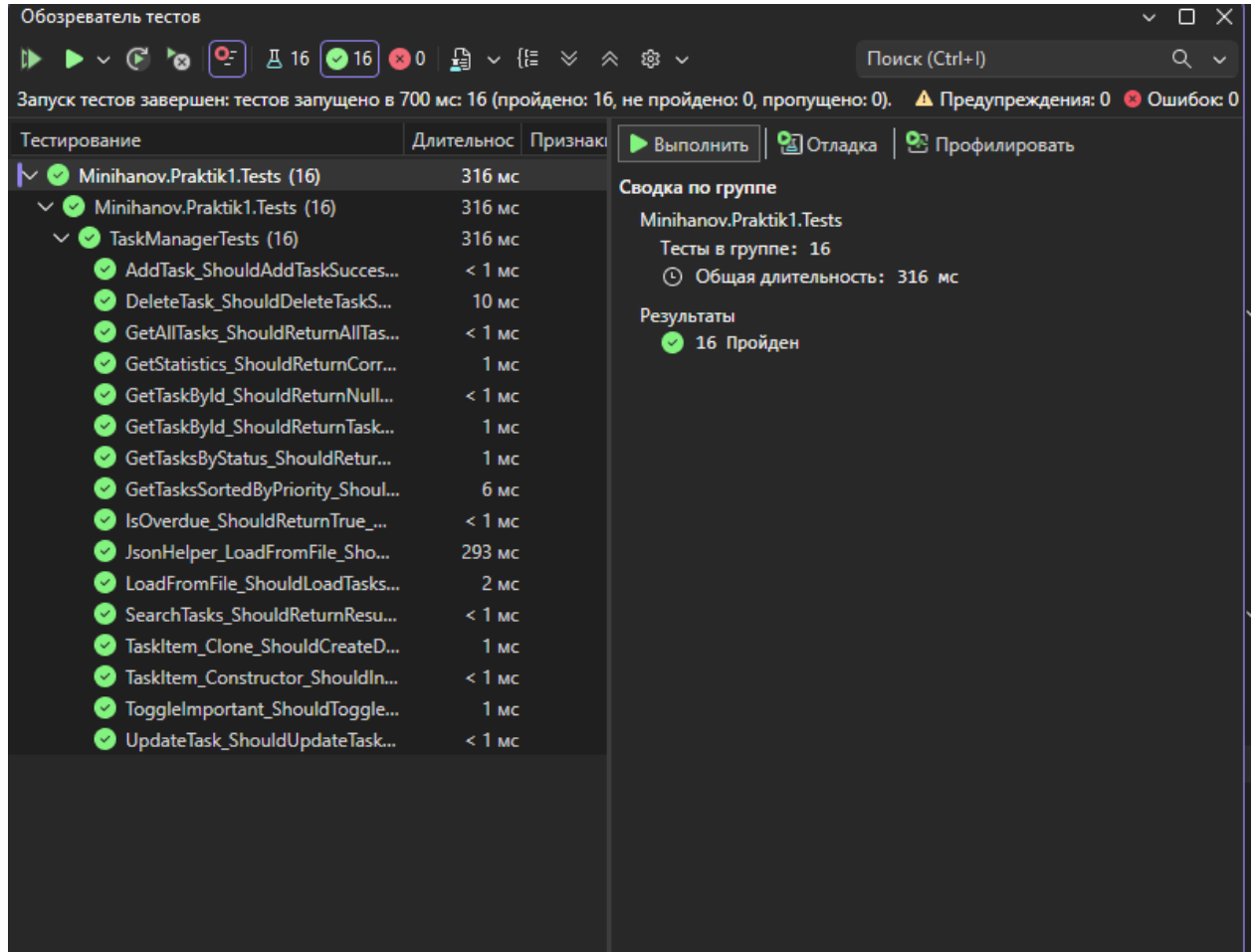


Рисунок 6 Результаты выполнения тестов

6.2. Возможные ошибки и исключения

В процессе разработки и отладки я столкнулся с несколькими ошибками — часть из них всплыла ещё на этапе написания кода, часть — при первом запуске. Расскажу о каждой подробно: что происходило, почему так вышло и как я это починил.

`NullReferenceException` в `RefreshTaskList()`

Самая первая и самая досадная ошибка. При запуске приложение падало с `NullReferenceException` в методе `RefreshTaskList()`. Оказалось, что я вызывал этот метод из конструктора `MainWindow` до того, как завершилась инициализация интерфейса (`InitializeComponent()`). В результате все UI-

элементы — кнопки, списки, фильтры — были ещё `null`, а метод пытался с ними работать.

Решение пришло быстро: я ввёл защитный флаг `_isInitialized`. Теперь перед любым вызовом `RefreshTaskList()` обработчики событий проверяют этот флаг, и если он `false` — метод просто не выполняется. В конструкторе я устанавливаю флаг в `true` сразу после `InitializeComponent()`, и только потом вызываю обновление списка. Ошибка ушла навсегда.

Некорректное определение просрочки

Следующая ошибка была не столь фатальна, но неприятна. Свойство `IsOverdue` в классе `TaskItem` определяло, просрочена задача или нет, сравнивая `DueDate` с `DateTime.Now`. Проблема в том, что `DateTime.Now` включает текущее время. Из-за этого задача, срок которой был, скажем, на сегодня, считалась просроченной уже в первой половине дня — ведь `DateTime.Now` могло быть больше, чем `DueDate` (если время задачи было, например, 23:59, а сейчас 10:00).

Я исправил логику: теперь сравнение идёт только по дате — `DueDate.Date < DateTime.Today`. Если срок выпадает на сегодняшний день, задача не считается просроченной до полуночи. Логично и справедливо.

`InvalidCastException` при разборе статуса

В методе обновления списка я пытался преобразовать строку из выпадающего списка фильтра в значение перечисления `TaskStatus` с помощью `Enum.Parse`. Но в списке был пункт «Все», который не соответствует ни одному члену перечисления. При попытке разобрать «Все» вылетало исключение.

Я добавил проверку: если выбранное значение равно «Все», то фильтрацию по статусу просто пропускаю. А для всех остальных строк использую безопасный `Enum.TryParse`. Теперь код работает корректно, даже если в фильтре что-то не так.

`KeyNotFoundException` при обновлении и удалении несуществующей задачи

Иногда случалось, что пользователь пытался изменить или удалить задачу, которую уже кто-то удалил (например, при параллельной работе — хотя у нас приложение однопользовательское, но такое могло произойти при некорректных данных). В сервисе методы `UpdateTask` и `DeleteTask` обращались к коллекции по индексу или `Id` без проверки существования, что приводило к `KeyNotFoundException`.

Я добавил в эти методы явную проверку на наличие задачи с переданным `Id`. Если такой задачи нет — я выбрасываю `KeyNotFoundException` с понятным сообщением: «Задача с идентификатором ... не найдена». Это позволило корректно обрабатывать ситуацию на уровне интерфейса и показывать пользователю информативное сообщение, а не «сырое» исключение.

Все эти ошибки были найдены в процессе отладки и тестирования. После исправлений приложение работает стабильно, без вылетов и неожиданного поведения.

Все операции, которые связаны с файлами (сохранение (`SaveToFile`) и загрузка (`LoadFromFile`)) — я обернул в блоки `try-catch`. Это значит, что если вдруг что-то пойдёт не так: файл занят, нет прав доступа, диск переполнен или данные повреждены, приложение не упадёт и не вылетит. Вместо этого пользователь увидит аккуратное модальное окно с понятным сообщением о том, что именно произошло. Такой подход гарантирует, что даже при возникновении ошибок программа останется стабильной и продолжит работу, а пользователь не потеряет свои данные и будет знать, что случилось.

Заключение

По итогам практики я разработал полноценное десктопное приложение «Планировщик задач» на C# с интерфейсом на WPF. Программа умеет всё, что нужно для удобного управления делами: добавлять, просматривать, редактировать и удалять задачи, искать по ключевым словам, фильтровать по статусу, сортировать по приоритету или сроку, а также сохранять данные в JSON-файл и загружать их обратно.

Архитектуру я строил так, чтобы код был понятным и гибким: вся бизнес-логика вынесена в отдельную библиотеку `Minihanov.Praktik.Core`, а взаимодействие между частями приложения идёт через интерфейс `ITaskService`. Это позволило не только разделить ответственность между слоями, но и сильно упростило тестирование — интерфейс можно проверять отдельно от интерфейса.

Чтобы убедиться, что всё работает как надо, я написал 16 модульных тестов — они покрывают все основные методы сервиса, логику модели данных и операции с файлами. В процессе отладки я нашёл и исправил четыре ошибки, включая проблемы с инициализацией окна и неверное определение просроченных задач.

В итоге приложение полностью соответствует тому, что было заявлено в техническом задании. Оно работает стабильно, удобно в использовании и показывает, как на практике применяются ключевые технологии и архитектурные подходы .NET-разработки.